Phase 1: Bronze Layer (Raw Ingestion)

The pipeline begins at the **Bronze** tier, representing the immutable vault of all raw, high-frequency telemetry emitted by the vehicle.

Data Structure: Bronze Table

Column Name

Type

Description

`timestamp`

Datetime

Exact millisecond of the sensor reading.

`source_id`

String

Unique vehicle identifier (e.g., `sim001`).

`row_hash`

String

Cryptographic hash for idempotency.

`[sensor_features]`

Float

Raw physical measurements (e.g., RPM, Temp, Pressure).

Phase 2: Silver Layer (Anomaly Detection & Enrichment)

The Inference Engine extracts a rolling window of data from Bronze to evaluate the health of a specific vehicle module (e.g., Engine, Transmission).

2.1 Imputation and Scaling

For a given module with D expected features, the raw data vector $x_t \in \mathbb{R}^D$ is evaluated. Missing values are imputed using Forward Fill, followed by replacement with historical feature means μ_{train}. The vector is then standardized using the parameters learned during offline training:

$$x'_t = \frac{x_t - \mu_{train}}{\sigma_{train}}$$

2.2 Sequence Generation

The models evaluate time-series context, not just point-in-time anomalies. The sequence of length T (where $T = 10$) is defined as:

$$X = [x'_{t-T+1}, x'_{t-T+2}, \ldots, x'_t]$$

2.3 LSTM Autoencoder (The Primary Anomaly Brain)

The input matrix X is fed into a Long Short-Term Memory Autoencoder (LSTM-AE).

1. **Encoder:** Compresses the sequence into a latent representation h_T.

$$h_\tau = \mathrm{LSTM}_{enc}(x'_\tau, h_{\tau-1}) \quad \forall \tau \in [t - T + 1, t]$$

2. **Decoder:** Attempts to reconstruct the sequence from the latent state.

$$\hat{x}'_\tau = \mathrm{LSTM}_{dec}(h_T)$$

3. **Raw Error Computation:** The anomaly magnitude is defined as the Mean Squared Error (MSE) between the actual and reconstructed values of the *final step* t:

$$L_{raw} = \frac{1}{D} \sum_{i=1}^{D} (\hat{x}'_{t,i} - x'_{t,i})^2$$

2.4 Gaussian Mixture Model (The Familiarity Check)

While the LSTM detects temporal disruptions, a Gaussian Mixture Model (GMM) evaluates the spatial likelihood of the current state. The GMM computes the log-probability density $LL(x'_t)$:

$$LL(x'_t) = \log \sum_{k=1}^{K} \pi_k \mathcal{N}(x'_t | \mu_k, \Sigma_k)$$

A "Familiarity Score" $\mathcal{F} \in [0, 1]$ is computed using training thresholds (p_{05} and min):

- If $LL \geq p_{05}$, $\mathcal{F} = 1.0$ (Highly familiar operating condition).
- If $LL < p_{05}$, the familiarity degrades linearly:

$$\mathcal{F} = \max\left(0, 1 - \frac{|p_{05} - LL|}{|p_{05} - min|}\right)$$

2.5 Error Smoothing (EMA)

To prevent sensor noise from causing erratic health scores, L_{raw} is smoothed using an Exponential Moving Average (EMA) with decay factor α:

$$L_{EMA,t} = \alpha \cdot L_{raw,t} + (1 - \alpha) \cdot L_{EMA,t-1}$$

2.6 Composite Score & Health Score Computation

The smoothed error $L_{EMA,t}$ is mapped to a normalized risk probability, the **Composite Score** $C \in [0, 1]$, using piecewise linear interpolation based on statistical boundaries derived from the training set ($p_{95}, p_{99.5}, max$):

1. **Normal Range:** If $L_{EMA} \leq p_{95}$

$$C = 0.5 \times \frac{L_{EMA}}{p_{95}}$$

2. **Warning Range:** If $p_{95} < L_{EMA} \leq p_{99.5}$

$$C = 0.5 + 0.3 \times \frac{L_{EMA} - p_{95}}{p_{99.5} - p_{95}}$$

3. **Critical Range:** If $L_{EMA} > p_{99.5}$

$$C = \min\left(1.0,\ 0.8 + 0.2 \times \frac{L_{EMA} - p_{99.5}}{max - p_{99.5}}\right)$$

Heuristic Adjustments using GMM:

- *Suppression:* If $\mathcal{F} > 0.8$ (highly familiar data) but $C < 0.9$, the anomaly is likely benign. C is penalized: $C = C \times 0.6$.
- *Amplification:* If $\mathcal{F} < 0.2$ (highly unfamiliar domain state), C is boosted: $C = \min(C + 0.2, 1.0)$.

Final Health Score:

$$Health_{module} = (1 - C) \times 100$$

2.7 Top-K Feature Attribution

To explain the anomaly, the system calculates the localized reconstruction error for each feature i:

$$Err_i = (\hat{x}'_{t,i} - x'_{t,i})^2$$

The features are sorted descending by Err_i, and the top 8 are stored as JSON for root cause tracking.

Data Structure: Silver Table

Column Name

Type

Description

`[bronze_columns]`

Mixed

100% preservation of raw Bronze telemetry.

`lstm_raw_error`

Float

L_{raw} at time t.

`lstm_smoothed`

Float

L_{EMA} at time t.

`composite_score`

Float

$C \in [0, 1]$.

`health_score`

Float

$Health \in [0, 100]$.

`severity`

String

NORMAL, WARNING, or CRITICAL.

`top_features`

JSON

Dict of `{"sensor_name": error_value}`.

Phase 3: Gold Layer (Vehicle Aggregation)

The Gold Vehicle state provides an executive summary of the entire asset.

3.1 Weighted System Health

The overall vehicle health is a weighted linear combination of all its constituent modules (where w_m represents the criticality weight of the module defined in the contract):

$$Health_{vehicle} = \sum_{m \in Modules} w_m \cdot Health_m$$

(e.g., Engine = 0.40, Tyre = 0.10).

Data Structure: Gold Vehicle Table

Column Name

Type

Description

`vehicle_health_score`

Float

Aggregate vehicle health ($0 - 100$).

`[module]_health`

Float

Individual health scores for engine, battery, etc.

Phase 4: Gold Alerts (State Machine Integration)

To prevent alert fatigue, a discrete state machine utilizing a **"Leaky Bucket"** algorithm is applied to the time-series flow of `severity` classifications from the Silver layer.

4.1 The Leaky Bucket Math

For each module, a persistent `fault_score` $F_t \in [0, 100]$ is maintained. For every incoming row, Δ is applied based on the Silver `severity`:

- If CRITICAL: $\Delta = +20$
- If WARNING: $\Delta = +5$
- If NORMAL: $\Delta = -10$

$$F_t = \max(0, \min(100, F_{t-1} + \Delta))$$

4.2 Alert Lifecycle Logic

- **Opening:** If F_t transitions from 0 to > 0, an Alert is generated with status `OPEN`.
- **Peak Tracking:** While the alert is OPEN, the engine continuously tracks the maximum Silver `composite_score` observed.

  $$C_{max} = \max(C_{max}, C_t)$$

  The exact timestamp where C_{max} is updated is locked in as `peak_anomaly_ts`.
- **Closing:** If F_t decays back down to 0 (due to consecutive NORMAL readings), the Alert is marked `CLOSED`, and C_{max} tracking is finalized.

Data Structure: Gold Alerts Table

Column Name

Type

Description

`alert_id`

String

Unique UUID.

`status`

String

OPEN or CLOSED.

`peak_anomaly_ts`

Datetime

The exact millisecond where $C_t == C_{max}$.

`max_composite_score`

Float

The absolute highest anomaly registered (C_{max}).

Phase 5: Diagnostic Deep Dive (DTC Analysis)

When an alert is flagged, the system executes an isolated, highly specialized Neural Network routine strictly over the period surrounding the `peak_anomaly_ts` to extract the Diagnostic Trouble Code (DTC).

5.1 Data Extraction

The system queries the **Bronze** layer and slices a sequence matrix X_{trace} consisting of the N rows (e.g., $N = 600$) ending exactly at the `peak_anomaly_ts`.

5.2 Specific Feature Slicing & Inference

For each DTC defined in the master contract (e.g., P0217 Engine Overheat), a strictly mapped subset of 4 features is extracted from X_{trace} and standardized.

$$x_{DTC,t} = \text{scaler}_{DTC}.transform(X_{trace,t})$$

A shallow Feed-Forward Neural Network (32 $\rightarrow$ 16 $\rightarrow$ 1) processes every row in the slice to output a raw fault probability P_t:

$$P_t = \sigma(W_2 \cdot \text{ReLU}(W_1 x_{DTC,t} + b_1) + b_2)$$

5.3 Smart Attribution Algorithm (Risk Accumulation)

To definitively isolate the true root cause from background mechanical noise, **Exponential Magnification** is applied across all active DTC probabilities in a unified global calculation.

1. **Dynamic Noise Floor Estimation:**

   For every DTC code c, the median probability over the N rows is established as its baseline:

   $$\beta_c = \text{median}(P_{c,1\ldots N})$$

2. **Exponential Magnification:**

   The positive evidence above the floor is extracted and **squared**. Squaring ensures that minor fluctuations (noise) become mathematically insignificant, while major sustained anomalies (the root cause) explode in magnitude.

   $$e_{c,t} = \max(0,\ P_{c,t} - \beta_c)$$

   $$M_{c,t} = (e_{c,t})^2$$

3. **Cumulative Risk Integration:**

The magnified evidence is integrated (summed) over time to represent the total fault buildup B_c:

$$B_{c,T} = \sum_{t=1}^{T} M_{c,t}$$

4. **Global Normalization (The Winner-Takes-All):**

   To convert these unbounded integrated scores into standard $0.0 - 1.0$ risk trajectories, the system identifies the absolute worst anomaly across *all* evaluated DTCs for that module:

$$B_{max} = \max_{c}(B_{c,N})$$

   A global scale factor is applied to all integration curves:

$$S_{c,T} = \min\left(1.0, \frac{B_{c,T}}{B_{max}}\right)$$

5.4 Diagnosis Threshold

By definition of the algorithm, the single DTC that accumulated the most magnified risk will terminate at exactly $S_{c,N} = 1.0$.

Any DTC curve where $S_c \geq 0.99$ is officially triggered, and its corresponding engineering definition (e.g., “CRITICAL: Coolant Overheat Detected”) is presented on the dashboard. All other secondary vibrations and warnings are visually and mathematically scaled down.